Web-приложения и их архитектура Front-end. Введение

Немного теории о фронте

Frontend часть. Что, зачем, почему (словно не знаете)

- Это всё, что пользователь видит и с чем взаимодействует
- Работает в браузере

Если быстро и своими словами — красивая обёртка, в которой пользователь может взаимодействовать с функционалом вашего приложения

Технологии веба

- HTML — структура
- CSS — стили
- JavaScript — логика

HTML/CSS

Что такое HTML?

HTML (HyperText Markup Language) — это язык разметки, который описывает структуру веб-страницы. Браузер читает HTML и отображает содержимое: текст, картинки, ссылки.

Минимальная структура HTML

Пояснения к коду:

`<!DOCTYPE html>` — говорит браузеру, что это HTML5

`<head>` — невидимая «шапка»: настройки, заголовок вкладки

`<body>` — всё, что видит пользователь

```html
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8">
    <title>Моя страница</title>
  </head>
  <body>
    <h1>Привет, мир!</h1>
    <p>Это моя первая страница.</p>
  </body>
</html>
```

Основные теги HTML

Тег	Назначение	Пример
`<h1>–<h6>`	Заголовки	`<h1>Главный заголовок</h1>`
`<p>`	Абзац	`<p>Текст абзаца</p>`
`<a>`	Ссылка	`<a href="https://google.com">Google</a>`
`<img>`	Изображение	`<img src="photo.jpg" alt="Фото">`
`<ul>/<li>`	Список	`<ul><li>Пункт</li></ul>`
`<div>`	Блок-контейнер	`<div>...</div>`
`<span>`	Строчный контейнер	`<span>слово</span>`

Атрибуты тегов

Атрибуты уточняют поведение тега. Они пишутся внутри открывающего тега. id — уникальный (один на странице), class — можно использовать много раз.

```
<!-- href — куда ведёт ссылка -->

<a href="https://example.com" target="_blank">Открыть сайт</a>

<!-- src — источник картинки, alt — текст если картинка не загрузилась -->

<img src="cat.jpg" alt="Рыжий кот" width="300">

<!-- class и id — для CSS и JavaScript -->

<p class="highlight" id="intro">Важный текст</p>
```

Что такое CSS?

CSS (Cascading Style Sheets) управляет тем, как выглядит HTML. Цвета, шрифты, отступы, расположение элементов — всё это CSS.

```
<!-- 1. Внешний файл (рекомендуется) -->

<link rel="stylesheet" href="style.css">

<!-- 2. Внутри тега <style> в <head> -->

<style> h1 { color: red; } </style>

<!-- 3. Прямо в теге (избегать!) -->

<p style="color: blue;">Текст</p>
```

Синтаксис CSS

```
h1 {
  color: #2c3e50;      /* цвет текста */
  font-size: 32px;     /* размер шрифта */
  text-align: center;  /* выравнивание */
}
p {
  font-family: Arial, sans-serif;
  line-height: 1.6;    /* межстрочный интервал */
}
```

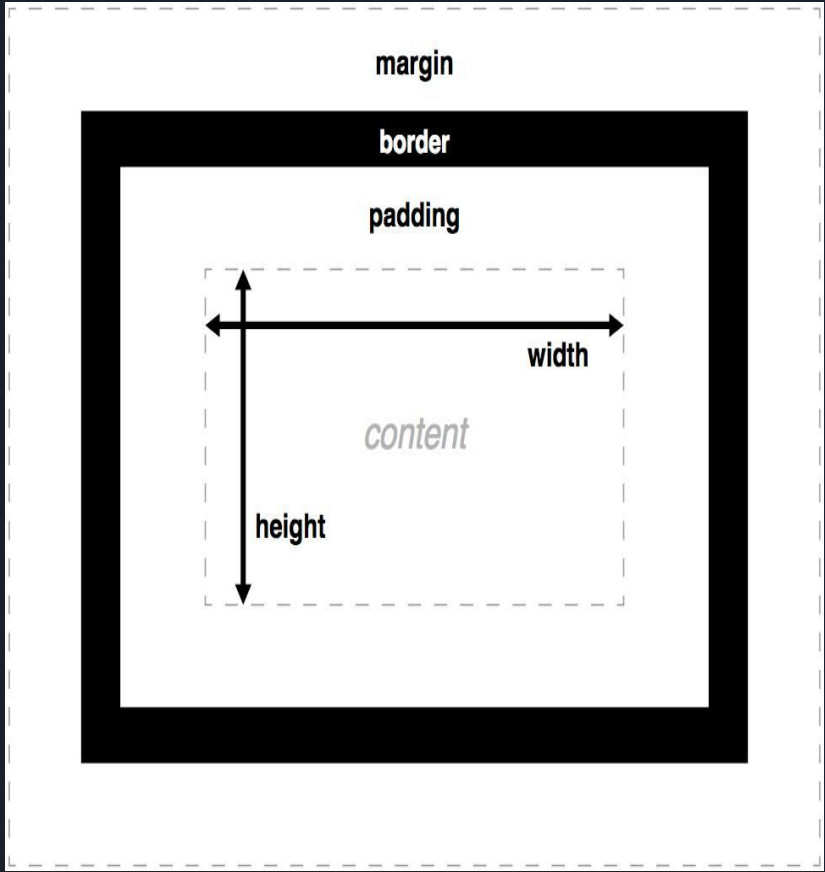
Селекторы CSS

Селектор	Синтаксис	Что выбирает
Тег	`p { }`	Все `<p>` на странице
Класс	`.highlight { }`	Все элементы с `class="highlight"`
ID	`#intro { }`	Элемент с `id="intro"`
Потомок	`div p { }`	Все `<p>` внутри `<div>`
Псевдокласс	`a:hover { }`	Ссылка при наведении мыши

Блочная модель

Любой HTML-элемент в браузере — прямоугольник, состоящий из 4 слоёв (изнутри наружу)

```
div {
  width: 200px; /* ширина содержимого */
  padding: 16px; /* отступ внутри */
  border: 2px solid black;
  margin: 24px; /* отступ снаружи */
}
```

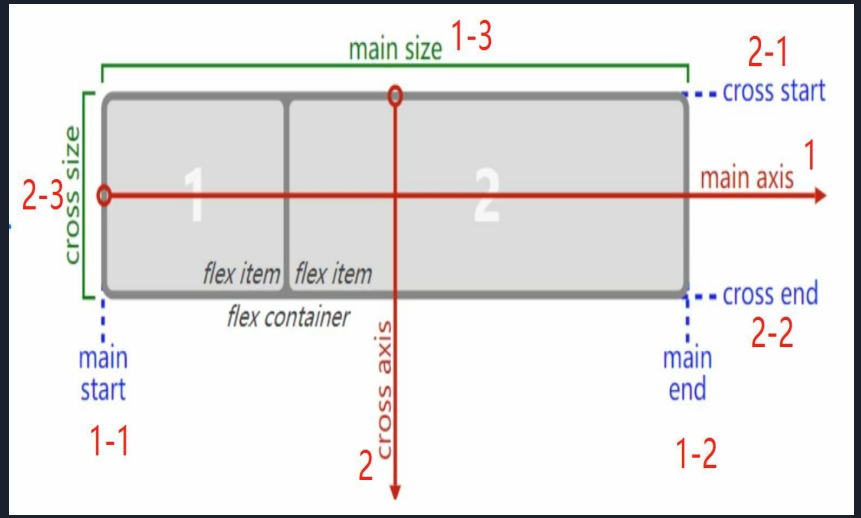


Display: block, inline, inline-block

Значение	Примеры тегов	Поведение
`block`	`div`, `p`, `h1`	Занимает всю ширину, начинается с новой строки
`inline`	`span`, `a`, `strong`	Занимает только нужное место, не переносится
`inline-block`	(задаётся вручную)	Как inline, но можно задавать width/height
`none`	—	Элемент скрыт и не занимает место

Flexbox — гибкое расположение элементов

```
.container {

        display: flex;

         justify-content: space-between; /* по горизонтали */

        align-items: center; /* по вертикали */

        gap: 16px; /* расстояние между элементами */

}
```



CSS Grid — двумерная сетка

Grid позволяет управлять и строками, и столбцами одновременно. Идеален для макетов страниц.

```
.grid { display: grid; grid-template-columns: 1fr 2fr 1fr; /* 3 колонки */ grid-template-rows: auto; gap: 20px; }
```

fr (fraction) — доля свободного пространства. 1fr 2fr 1fr = центральная колонка вдвое шире боковых.

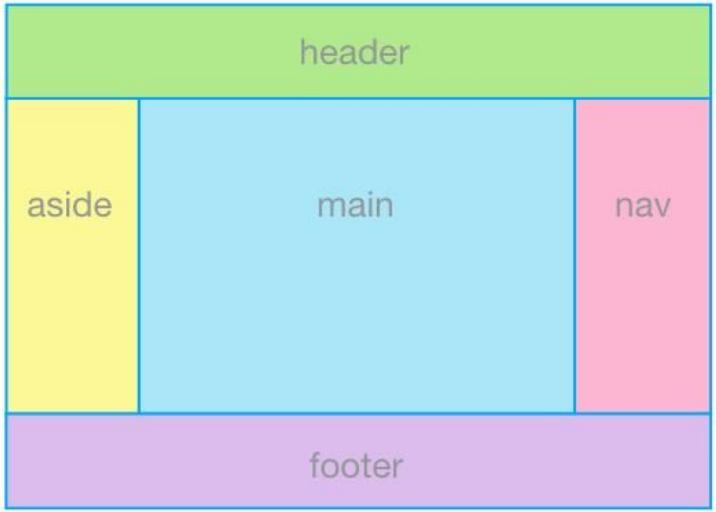
```
grid-template-columns: repeat(3, 1fr); /* 3 равные колонки */
```

Flexbox vs Grid: когда что использовать?

	Flexbox	Grid
Измерение	Одно (ряд или колонка)	Два (строки и колонки)
Лучше для	Навигации, карточек в ряд	Макета всей страницы
Контроль	Дети управляют собой	Родитель управляет всем

Несколько простых правил: Flexbox используем, если элемент расположены в одну линию. Сложный макет страницы - юзаем Grid. Можно использовать комбинированно.

Структура страницы



Небольшая музыкальная пауза

JS (ЖабаСкрЫпт)

JS - что это. Его особенности

JavaScript (**JS**) — это легковесный интерпретируемый (или JIT-компилируемый) язык программирования с функциями первого класса. Наиболее широкое применение находит как язык сценариев веб-страниц, но также используется и в других программных продуктах, например, Node.js или Apache CouchDB. JavaScript это прототипно-ориентированный, мультипарадигменный язык с динамической типизацией, который поддерживает объектно-ориентированный, императивный и декларативный (например, функциональное программирование) стили программирования.

Типы переменных. Типы данных

Способы объявить переменную:

1. var
2. let
3. const

Типы данных:

1. Boolean
2. null
3. undefined
4. Number
5. BigInt
6. Symbol
7. String
8. Object (не примитив, пока просто запомним)

Оператор сравнения. Строгое/нестрогое равенство

Оператор **нестрогого равенства (==)** проверяет, являются ли два его операнда равными, возвращая логическое значение. В отличие от оператора строгого равенства (===) пытается привести и сравнить операнды разных типов.

Пример:

```
console.log(1 == 1); // Expected output: true

console.log(“1” == 1) // Expected output: true

console.log(“1” === 1) // Expected output: false
```

Functions

Типичные функции. Так как JS - C-подобный язык, то и функции тут имеют точно такой же функционал. Пример:

```
function foo (name) { return `Hello ${name}` }

console.log(foo(“Jam”));
```

Function Expression vs Function Declaration

Function Declaration

```
greet(); // Works! Output: "Hello"

function greet() {

  console.log("Hello");

}
```

Function Expression

```
greet(); // ReferenceError: Cannot access
'greet' before initialization

const greet = function() {

  console.log("Hello");

};
```

Arrow Functions

Теперь рассмотрим ещё один крайне интересный вариант объявления функции: arrow function

Пример:

```
const foo = () => console.log(‘Hello from arrow func’);
```

Можно записать и в несколько строк:

```
const foo = () => {

      console.log(‘Hello from arrow func’);

}
```

Object

Тип **Object** представляет один из типов данных JavaScript. Он используется для хранения различных коллекций с ключами и более сложных сущностей. Объекты могут быть созданы с использованием конструктора Object() или синтаксиса инициализатора / литерала объекта.

Почти все объекты в JavaScript являются экземплярами Object; типичный объект наследует свойства (включая методы) от Object.prototype, хотя эти свойства могут быть затенены (т.е. переопределены). Единственные объекты, которые не наследуют от Object.prototype, - это те, у которых прототип null, или которые происходят от других объектов с прототипом null.

Прототипно-ориентированное программирование

Прототипно-ориентированный стиль объектно-ориентированного программирования тот, в котором классы определены неявно, а производные классы получаются добавлением свойств и методов в экземпляр другого класса или, значительно реже, к пустому объекту.

На сегодня пока всё. Опять-таки, есть время на вопросы и поболтать